PROVIDE AN **ABSTRACTION LADDER**

```ts
export function mountD3(host: HTMLElement) {
  const w = 600, h = 300, m = 32;

  const x = d3.scalePoint()
    .domain(labels).range([m, w - m]);
  const y = d3.scaleLinear()
    .domain([0, Math.max(...data)]).range([h - m, m]);
  const line = d3.line<number>().x((_, i) => x(labels[i])!).y((d) => y(d));

  const svg = d3.select(host)
    .append("svg").attr("viewBox", `0 0 ${w} ${h}`);

  svg
    .append("path")
    .attr("fill", "none")
    .attr("stroke", "#2563eb")
    .attr("stroke-width", 2)
    .attr("d", line(data));
  svg
    .append("g")
    .attr("transform", `translate(0,${h - m})`)
    .call(d3.axisBottom(x));
  svg
    .append("g")
    .attr("transform", `translate(${m},0)`)
    .call(d3.axisLeft(y));
}
```

```
export function mountPlot(host: HTMLElement) {

  const chart = Plot.plot({
    marks: [Plot.lineY(points, {
      x: "label", y: "value", stroke: "#2563eb"
    })],
  });

  host.append(chart);
}
```


D3

Observable Plot


A


A

```
const w = 600, h = 300, m = 32;
```

```
const svg = d3.select(host)
```

```
const x = d3.scalePoint()
```

```
.append("path")
```

```
export function mountD3(host: HTMLElement) {
```

```
const line = d3.line<number>().x((_, i) ⇒ x(labels[i])!).y((d) ⇒ y(d));
```

```
.append("svg").attr("viewBox", `0 0 ${w} ${h}`);
```

```
const y = d3.scaleLinear()
```


svg

```
.attr("fill", "none")
```


.append("g")

```
.call(d3.axisBottom(x));
```

```
.attr("stroke", "#2563eb")
```

```
.attr("stroke-width", 2)
```

```
.domain([0, Math.max(...data)]).range([h - m, m]);
```

```
.attr("transform", `translate(${m},0)`)
```

```
.call(d3.axisLeft(y));
```

```
.attr("transform", `translate(0,${h - m})`)
```

```
.attr("d", line(data));
```

```
.domain(labels).range([m, w - m]);
```

```
marks: [Plot.lineY(points, {
```

```
export function mountPlot(host: HTMLElement) {
```

```
host.append(chart);
```

```
x: "label", y: "value", stroke: "#2563eb"
```

```
const chart = Plot.plot({
```

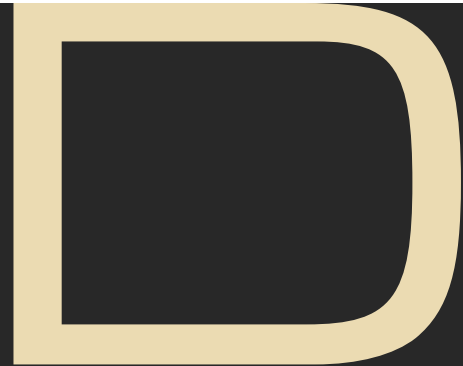
In my dream, there's a ladder of abstraction. On the lowest rung, low-level code for constructing

for data with overviews, zoom and filter, details on demand...

bespoke views of data — **WebGL fragment shaders**, say. At the highest rung, **visual interfaces**

November 19, 2021

Future of data work: Q&A with Mike Bostock




A


A